

Abstract digital background featuring mathematical formulas, binary code, and colorful bokeh effects. The formulas include:

- $1+x+y+2a+21$
- $1 \lim h \rightarrow 0$
- $x=0 \ x_n \{x-12-y+n\}$
- $x=0 \ x_n (1+x_0 y+2a+21) (g+x)$
- $x=0 \ x_n$
- $(1+x+y+2a) \cdot (3)$
- $x_n 5+x+k+2a+2$
- $(1+x+y+2a) \cdot$

The background is a vibrant mix of blue, green, and red, with a dense pattern of binary code and mathematical symbols. The overall effect is a complex, layered digital landscape.

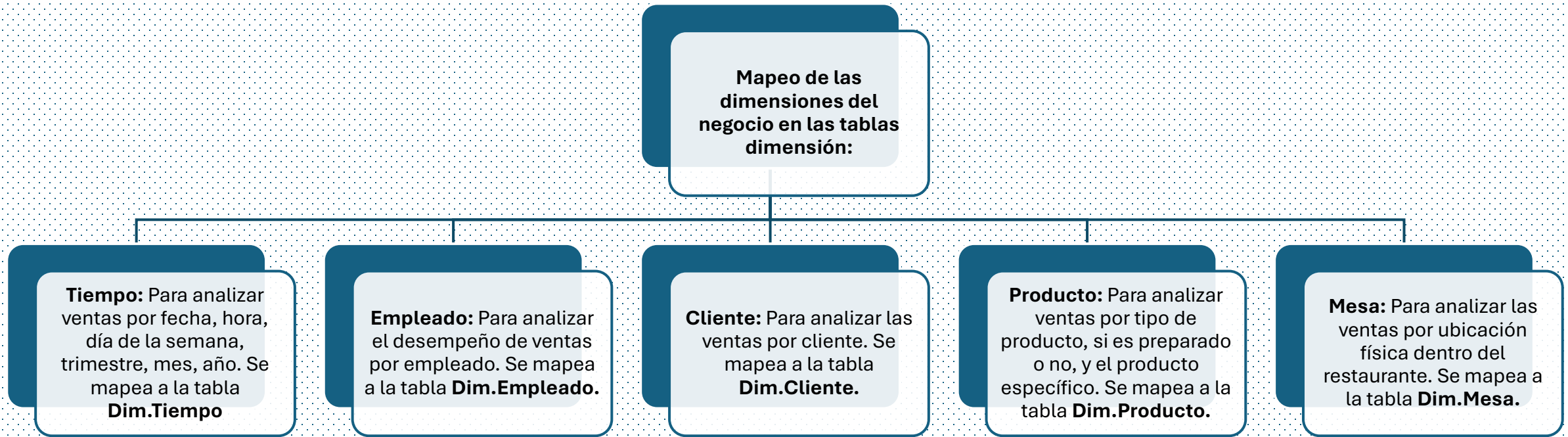
- **Integrantes:**
- Limay Rodriguez, Adriana Anthonela
- Llanos Cerquin, Melanie Brizeth
- Perez Briceño, Darick André
- Valdiviezo Zavaleta, Jesús Arturo

CAP 4. MODELO LÓGICO MODELO DIMENSIONAL



4.1 Desarrollo de un DATAMART para el área de Ventas de la empresa El sabor Cajamarquina.

4.2 Definir dimensiones.



- **Diccionario de datos**

DICCIONARIO DE LA DIMENSIÓN TIEMPO			
Atributo	Tipo de dato	Restricción	Descripción
tiempokey	INT	PRIMARY KEY, NOT NULL	Clave única de la dimensión
fecha	DATETIME	NOT NULL	La fecha completa en formato DATE
anio	INT	NOT NULL	El año
trimestre	INT	NOT NULL	El número del trimestre (1-4).
nombreTrimestre	CHAR(2)	NOT NULL	El nombre del trimestre (e.g., 'Q1')
mes	INT	NOT NULL	El número del mes (1-12).
nombreMes	NVARCHAR(50)	NOT NULL	El nombre completo del mes (e.g., 'Enero', 'Febrero').
semanaAnio	INT	NOT NULL	El número de la semana en el año (1 53).
diaAnio	INT	NOT NULL	El número del día del año
diaMes	INT	NOT NULL	El número del día del mes
diaSemana	INT	NOT NULL	El número del día de la semana (1=Lunes, 7=Domingo).
nombreDia	NVARCHAR(50)	NOT NULL	El nombre completo del día de la semana (e.g., 'Lunes', 'Martes').

- **Diccionario de datos**

DICCIONARIO DE LA DIMENSIÓN PRODUCTO			
Atributo	Tipo de dato	Restricción	Descripción
productokey	INT	PRIMARY KEY, NOT NULL	Clave única de la dimensión
categoría	NVARCHAR (50)	NOT NULL	categoría a la que pertenece el producto (Bebidas, Platos típicos, Platos criollos, Combinados, Entradas, Sopas, Postres, Fuentes).
tipo	BIT	NOT NULL	Es preparado (1) o no es preparado (0)
precio	DECIMAL(10,2)	NOT NULL	Precio del producto
nombreProducto	NVARCHAR(200)	NOT NULL	Nombre del producto

- **Diccionario de datos**

DICCIONARIO DE LA DIMENSIÓN EMPLEADO			
Atributo	Tipo de dato	Restricción	Descripción
empleadokey	INT	PRIMARY KEY, NOT NULL	Clave única de la dimensión
nombre	NVARCHAR (200)	NOT NULL	Nombre completo del empleado
fechacontratacion	DATE	NOT NULL	Fecha de contratación del empleado

- **Diccionario de datos**

DICCIONARIO DE LA DIMENSIÓN MESA			
Atributo	Tipo de dato	Restricción	Descripción
mesakey	INT	PRIMARY KEY, NOT NULL	Clave única de la dimensión y número de la mesa
ubicacion	NVARCHAR (50)	NOT NULL	Primer piso, segundo piso
capacidad	INT	NOT NULL	Número de sillas
estado	NVARCHAR(20)	NOT NULL	Disponibile, Fuera de servicio, reservada, ocupada

Asignación de claves primarias a cada dimensión

- **Dim_Tiempo**: tiempokey (PK)
- **Dim_Empleado**: empleadokey (PK)
- **Dim_Producto**: productokey (PK)
- **Dim_Cliente**: clientekey (PK)
- **Dim_Mesa**: mesakey (PK)

Identificando jerarquías analíticas

- **DIM.Tiempo**: Fecha → Año → Trimestre → Mes → Semana del Año → Día
- **DIM.Producto**: Categoría principal → Tipo de Producto → Nombre de producto → Precio
- **DIM.Cliente**: Nombre cliente
- **DIM.Empleado**: Nombre → fecha de contratación
- **DIM.Mesa** Ubicación → Numero → Capacidad → Estado
- **FacVentas** Ingresos Venta → Costo → Margen

Determinar la granularidad de cada dimensión

- **Dim_Tiempo:** La granularidad es por día. Cada fila representa un día único.
- **Dim_Producto:** La granularidad es por producto específico. Cada fila representa un producto único.
- **Dim_Cliente:** La granularidad es por cliente individual. Cada fila representa un cliente único.
- **Dim_Empleado:** La granularidad es por empleado individual. Cada fila representa un empleado único.
- **Dim_Mesa:** La granularidad es por mesa individual. Cada fila representa una mesa única.



Definir la tabla de hechos

Medidas del negocio en las tablas de hechos

- Ingresos totales
- Número de ventas
- Venta completa
- Costo
- Margen



Granularidad de la tabla de hechos



Ingresos venta: Se puede ver los ingresos totales por día, por mes, por tipo de producto, por empleado, por cliente, etc.



Número de ventas: Se puede contar el número de pedidos por día, por mes, por tipo de pedido, por empleado, etc.



Venta completa: Se puede ver las ventas completadas por día, por mes, por tipo de producto, por empleado, por cliente, etc.

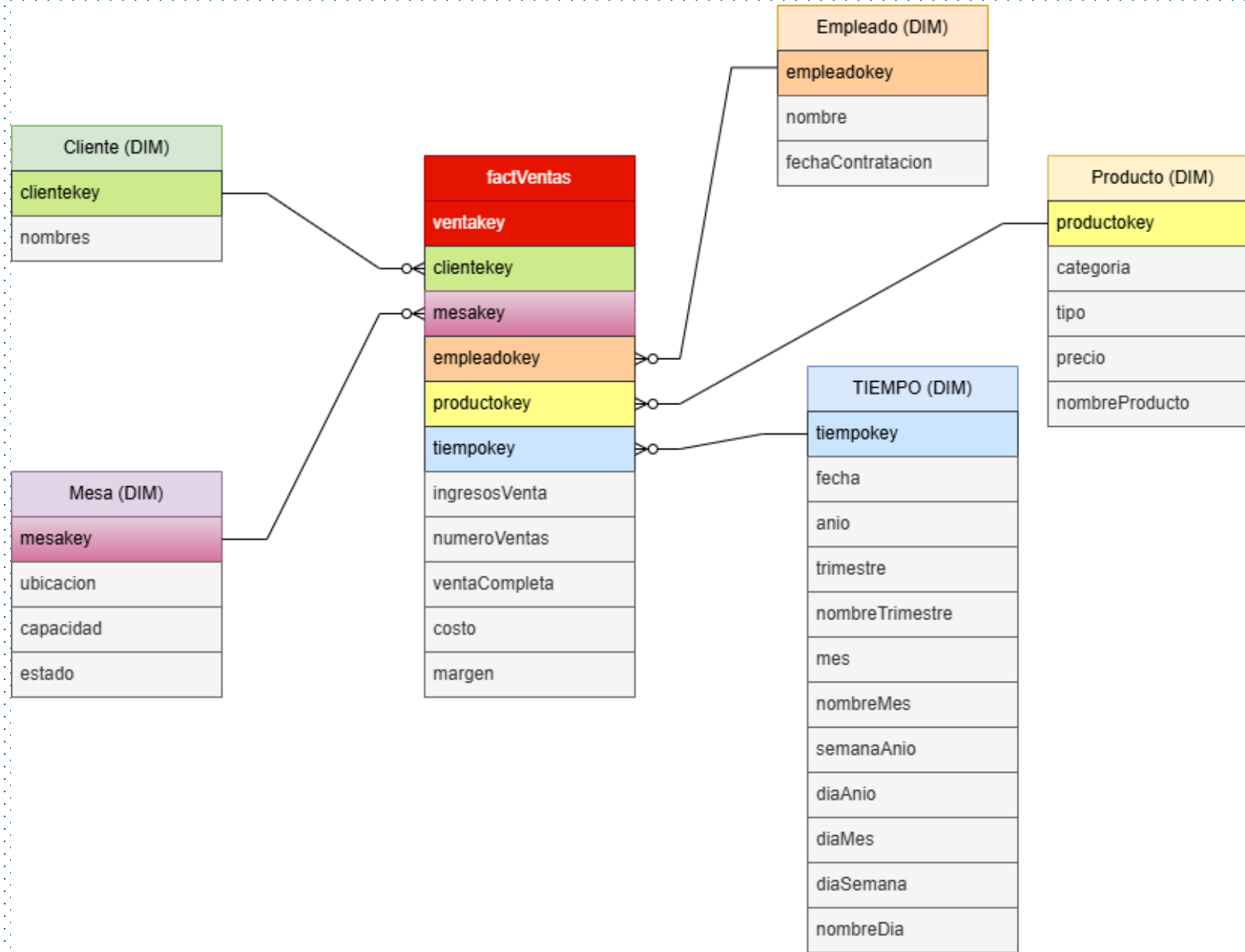


Costo: permite visualizar el costo asociado a los productos vendidos.

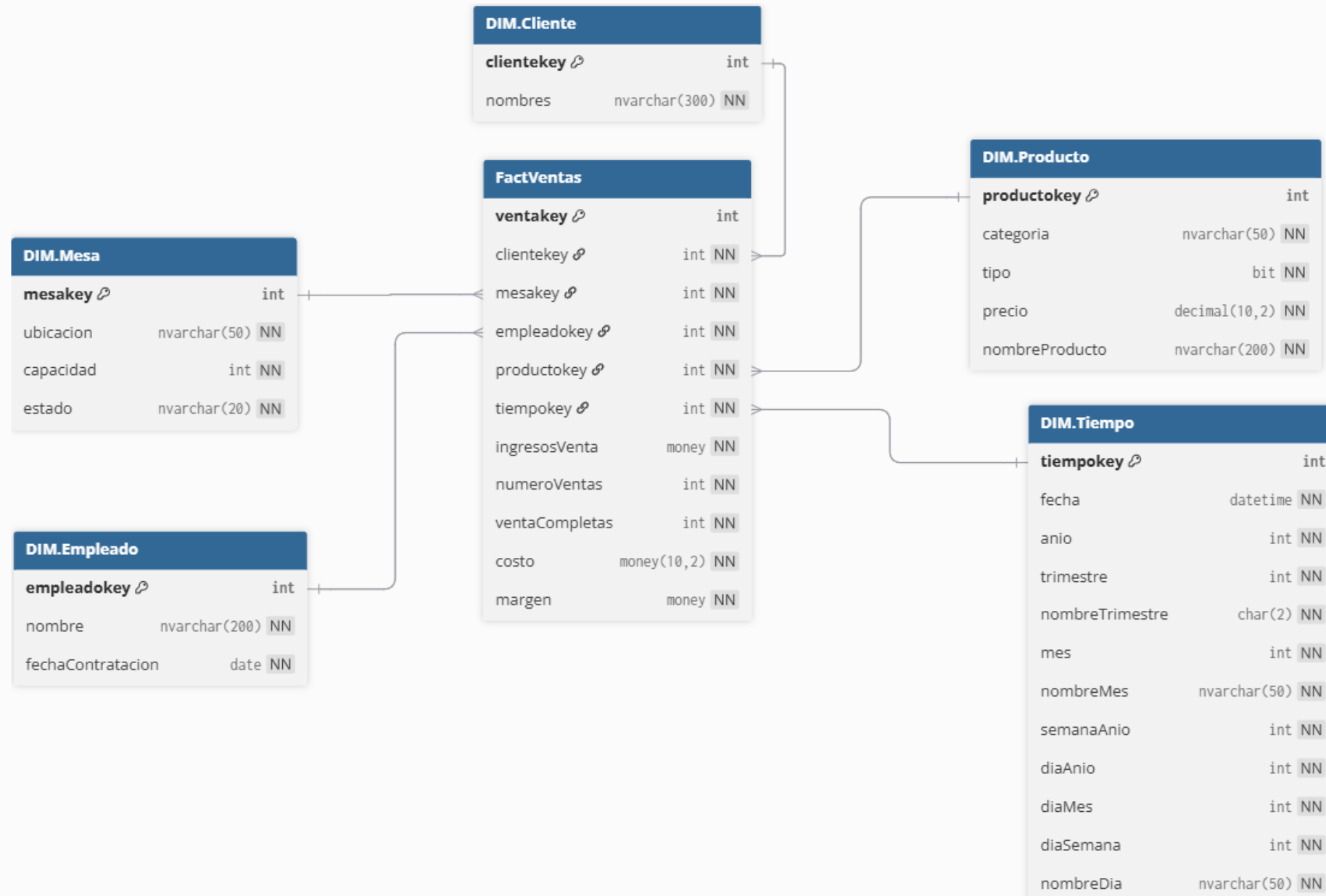


Margen: Permite calcular la diferencia entre el precio de venta y el costo (precio – costo).

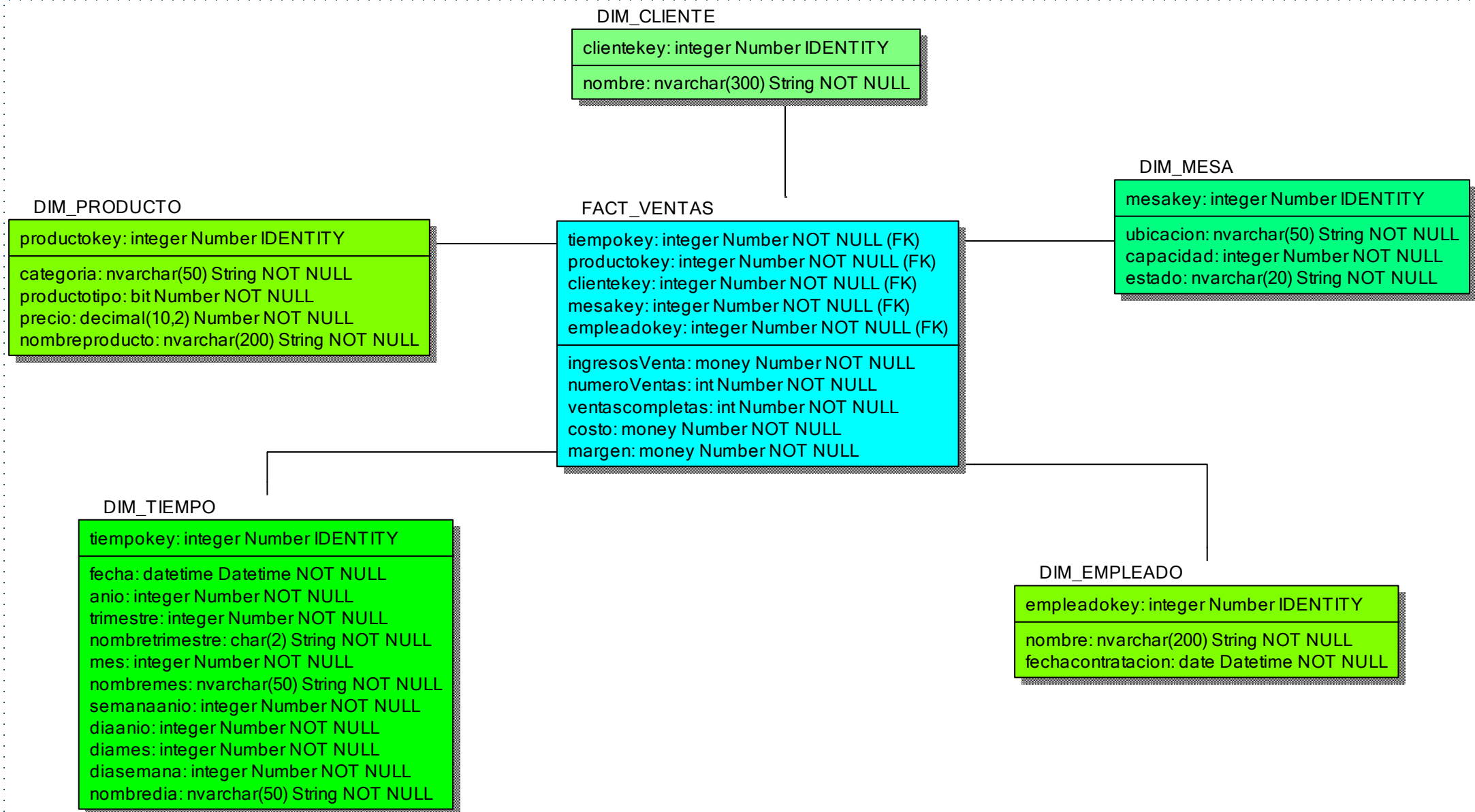
Modelo dimensional estrella o copo de nieve



Diseño físico modelo dimensional



Diseño físico modelo dimensional



Diseño físico modelo dimensional

Dimensión	Column Name	Column Datatype	Column Null Option	Column Is PK	Column Is FK
DIM.Cliente	clientekey	int	IDENTITY	Yes	No
	nombres	nvarchar(300)	NOT NULL	No	No
DIM.Mesa	mesakey	int	IDENTITY	Yes	No
	ubicación	nvarchar(50)	NOT NULL	No	No
	capacidad	int	NOT NULL	No	No
	estado	nvarchar(20)	NOT NULL	No	No
DIM.Producto	productokey	int	IDENTITY	Yes	No
	categoría	nvarchar(50)	NOT NULL	No	No
	tipo	bit	NOT NULL	No	No
	precio	decimal(10,2)	NOT NULL	No	No
	nombreProducto	nvarchar(200)	NOT NULL	No	No
DIM.Empleado	empleadokey	int	IDENTITY	Yes	No
	nombre	nvarchar(200)	NOT NULL	No	No
	fechaContratacion	date	NOT NULL	No	No
DIM.Tiempo	tiempokey	int	NOT NULL	Yes	No
	fecha	datetime	NOT NULL	No	No
	anio	int	NOT NULL	No	No
	trimestre	int	NOT NULL	No	No
	nombreTrimestre	char(50)	NOT NULL	No	No
	mes	int	NOT NULL	No	No
	nombreMes	nvarchar(50)	NOT NULL	No	No
	semanaAnio	int	NOT NULL	No	No
	díaAnio	int	NOT NULL	No	No
	díaMes	int	NOT NULL	No	No
	díaSemana	int	NOT NULL	No	No
	nombreDía	nvarchar(50)	NOT NULL	No	No
TablaHechos	ventakey	int	IDENTITY	Yes	Yes
	clientekey	int	NOT NULL	Yes	Yes
	mesakey	int	NOT NULL	Yes	Yes
	empleadokey	int	NOT NULL	Yes	Yes
	productokey	int	NOT NULL	Yes	Yes
	tiempokey	int	NOT NULL	Yes	Yes
	ingresosVenta	money	NOT NULL	No	No
	numeroVentas	int	NOT NULL	No	No
	ventaCompleta	int	NOT NULL	No	No
	costo	money	NOT NULL	No	No
	margen	money	NOT NULL	No	No

Creación de la base de datos dimensional

```
CREATE DATABASE RestauranteMart;  
USE RestauranteMart;  
CREATE SCHEMA DIM;
```

```
CREATE TABLE DIM.Tiempo(  
    tiempokey INT PRIMARY KEY IDENTITY(1,1),  
    fecha DATETIME NOT NULL,  
    anio INT NOT NULL,  
    trimestre INT NOT NULL,  
    nombreTrimestre CHAR(2) NOT NULL,  
    mes INT NOT NULL,  
    nombreMes NVARCHAR(50) NOT NULL,  
    semanaAnio INT NOT NULL,  
    diaAnio INT NOT NULL,  
    diaMes INT NOT NULL,  
    diaSemana INT NOT NULL,  
    nombreDia NVARCHAR(50) );
```

```
CREATE TABLE DIM.Cliente(  
    clientekey INT PRIMARY KEY IDENTITY(1,1),  
    nombres NVARCHAR(300) NOT NULL  
);
```

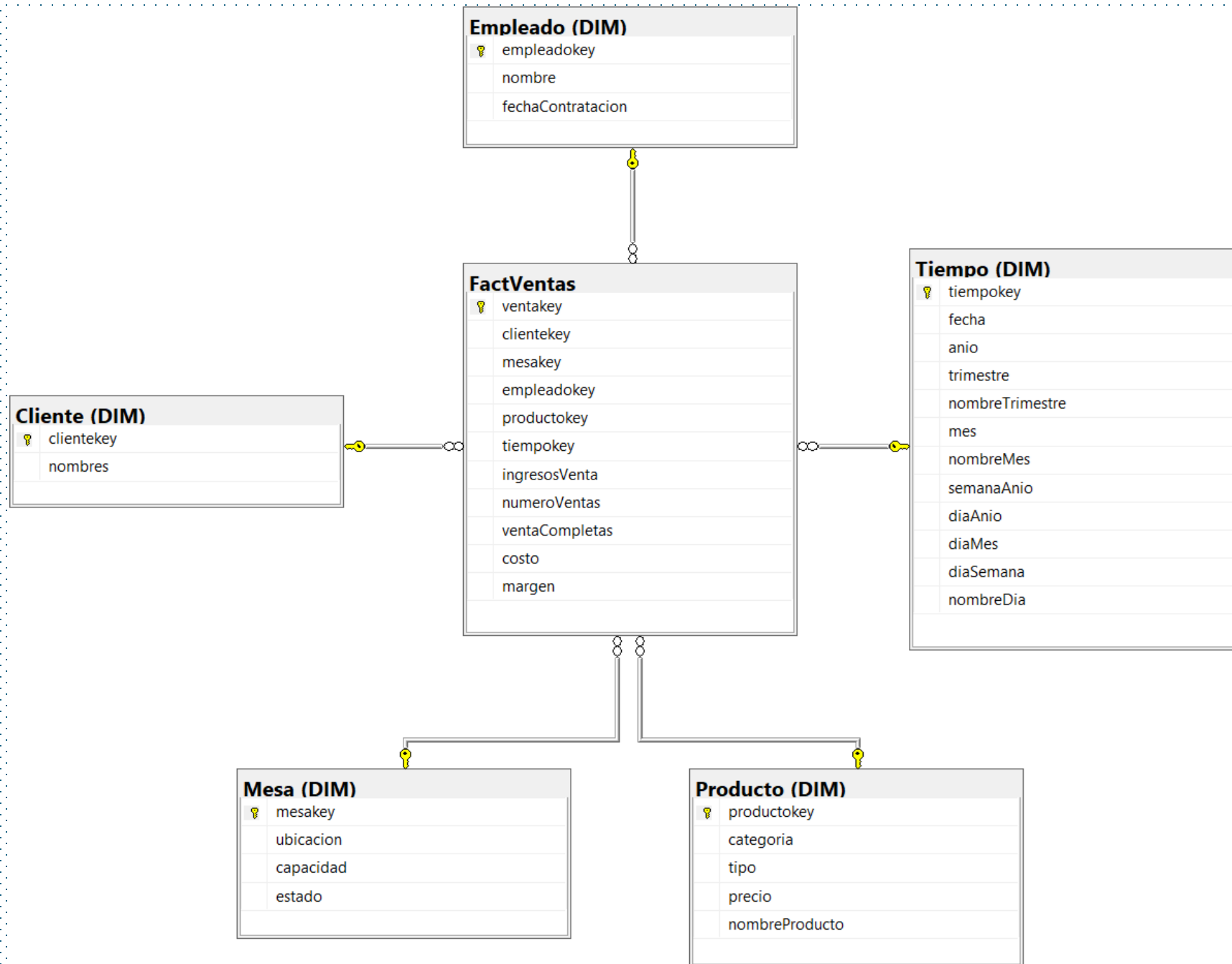
```
CREATE TABLE DIM.Mesa(  
    mesakey INT PRIMARY KEY IDENTITY(1,1),  
    ubicacion NVARCHAR(50) NOT NULL,  
    capacidad INT NOT NULL,  
    estado NVARCHAR(20) NOT NULL  
);
```

```
CREATE TABLE DIM.Empleado(  
    empleadokey INT PRIMARY KEY IDENTITY(1,1),  
    nombre NVARCHAR(200) NOT NULL,  
    fechaContratacion DATE NOT NULL);
```

```
CREATE TABLE DIM.Producto(  
    productokey INT PRIMARY KEY IDENTITY(1,1),  
    categoria NVARCHAR(50) NOT NULL,  
    tipo BIT NOT NULL,  
    precio DECIMAL (10,2) NOT NULL,  
    nombreProducto NVARCHAR(200) NOT NULL);
```

Creación de la base de datos dimensional

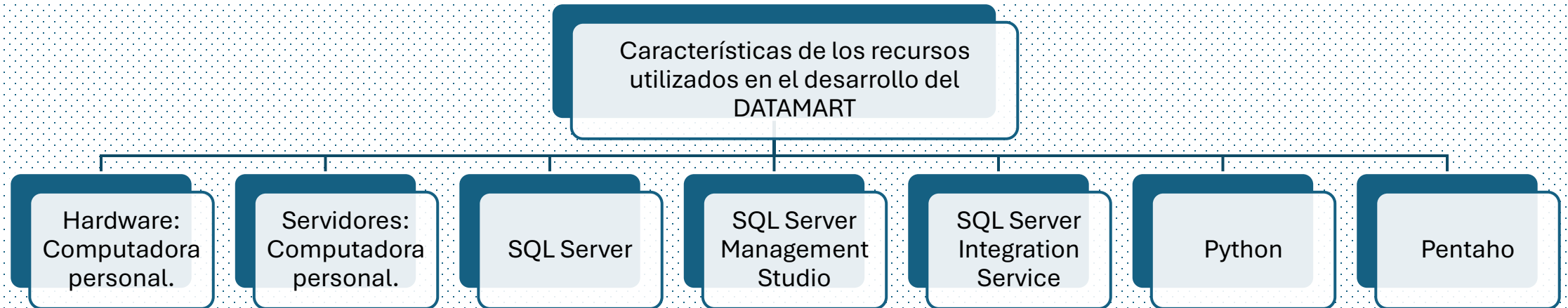
```
CREATE TABLE FactVentas(  
    ventakey INT PRIMARY KEY IDENTITY(1,1),  
    clientekey INT NOT NULL,  
    mesakey INT NOT NULL,  
    empleadokey INT NOT NULL,  
    productokey INT NOT NULL,  
    tiempokey INT NOT NULL,  
    ingresosVenta MONEY NOT NULL,  
    numeroVentas INT NOT NULL,  
    ventaCompletas INT NOT NULL,  
    costo MONEY NOT NULL,  
    margen MONEY NOT NULL,  
    CONSTRAINT clienteFK FOREIGN KEY (clientekey) REFERENCES DIM.Cliente(clientekey),  
    CONSTRAINT mesaFK FOREIGN KEY (mesakey) REFERENCES DIM.Mesa(mesakey),  
    CONSTRAINT empleadoFK FOREIGN KEY (empleadokey) REFERENCES DIM.Empleado(empleadokey),  
    CONSTRAINT productoFK FOREIGN KEY (productokey) REFERENCES DIM.Producto(productokey),  
    CONSTRAINT tiempoFK FOREIGN KEY (tiempokey) REFERENCES DIM.Tiempo(tiempokey)  
);
```

CAP 5. PROCESO DE EXTRACCION, TRANSFORMACIÓN Y CARGA DE DATOS (ETL)



Diseño de la arquitectura (Infraestructura: Servidores, Equipos)



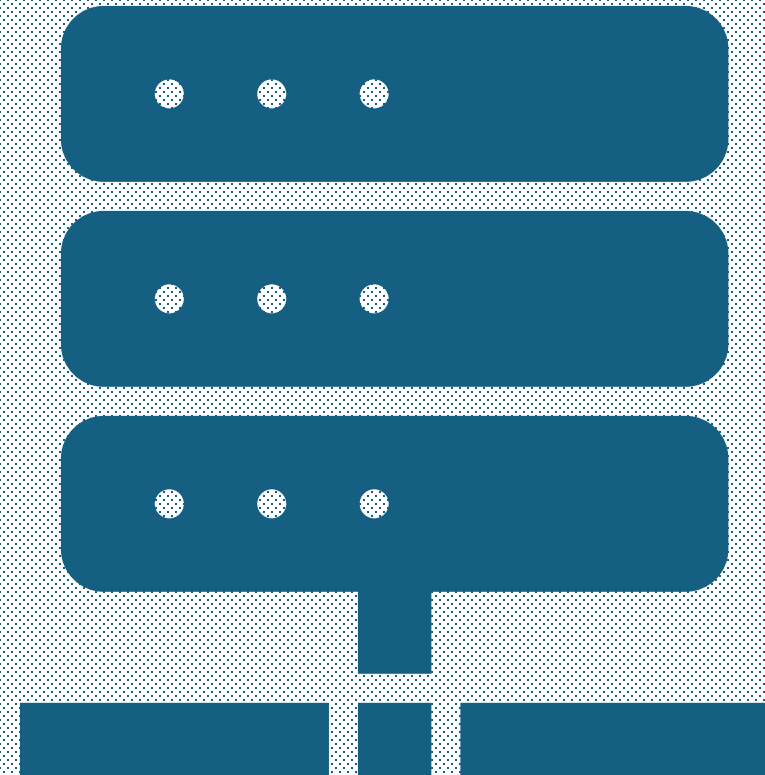
Diseño de la arquitectura (Infraestructura: Servidores, Equipos)

Características de los recursos
utilizados en la implementación del
DATAMART

- Hardware: Computadora personal.
- Servidores: Computadora personal.
- SQL Server Analysis Services
- Power BI

Características de los recursos
utilizados en la implementación del
DATAMART

- Hardware: Computadora personal.
- Servidores: Computadora personal.
- SQL Server Analysis Services
- Power BI



Extracción

Origen fuente: RestauranteDBasePrueba5m

Destino: RestauranteMart

--## 1. CONSULTA PARA DIM.Cliente

```
SELECT (Nombres + ' ' + ApellidoPaterno + ' ' +  
ApellidoMaterno) AS nombres  
FROM CLIENTE.Cliente  
ORDER BY 1;
```

--## 2. CONSULTA PARA DIM.Mesa

```
SELECT Ubicacion AS ubicacion,  
       Capacidad AS capacidad,  
       Estado AS estado  
FROM GENERAL.Mesa;
```

--## 3. CONSULTA PARA DIM.Empleado

```
SELECT NombreCompleto AS nombre,  
       FechaContratacion AS fechaContratacion  
FROM PERSONAL.Empleado
```

--## 5. CONSULTA PARA DIM.Tiempo

```
SELECT DISTINCT  
       Fecha AS fecha,  
       YEAR(Fecha) AS anio,  
       DATEPART(QUARTER, Fecha) AS trimestre,  
       CASE DATEPART(QUARTER, Fecha)  
         WHEN 1 THEN 'Q1'  
         WHEN 2 THEN 'Q2'  
         WHEN 3 THEN 'Q3'  
         WHEN 4 THEN 'Q4'  
       END AS nombreTrimestre,  
       MONTH(Fecha) AS mes,  
       DATENAME(MONTH, Fecha) AS nombreMes,  
       DATEPART(WEEK, Fecha) AS semanaAnio,  
       DATEPART(DAYOFYEAR, Fecha) AS diaAnio,  
       DATEPART(DAY, Fecha) AS diaMes,  
       DATEPART(WEEKDAY, Fecha) AS diaSemana,  
       DATENAME(WEEKDAY, Fecha) AS nombreDia  
FROM TRANSACCION.Pedido  
ORDER BY 1
```

Consulta multitabla para la tabla de hechos

--## 6. CONSULTA PARA FactVentas

```
WITH ProductoCosto AS (  
    SELECT  
        p.Id_Producto AS productoid,  
        p.EsPreparado,  
        ISNULL(  
            SUM(CASE  
                WHEN p.EsPreparado = 1 THEN ISNULL(inv_ing.CostoPorUnidad, 0) * ISNULL(pg_ing.Cantidad, 0)  
                ELSE 0  
            END),  
            0  
        ) AS CostoCalculadoIngredientes,  
        ISNULL(  
            MAX(CASE  
                WHEN p.EsPreparado = 0 THEN ISNULL(inv_direct.CostoPorUnidad, 0)  
                ELSE 0  
            END),  
            0  
        ) AS CostoDirectoProducto  
    FROM [RestauranteDBasePrueba5m].[GENERAL].[Producto] p  
    LEFT JOIN [RestauranteDBasePrueba5m].[GENERAL].[ProductoIngrediente] pg_ing ON p.Id_Producto = pg_ing.Id_Producto  
    LEFT JOIN [RestauranteDBasePrueba5m].[INVENTARIO].[Inventario] inv_ing ON pg_ing.Id_Item = inv_ing.Id_Item  
    LEFT JOIN [RestauranteDBasePrueba5m].[INVENTARIO].[Inventario] inv_direct ON p.Nombre = inv_direct.ItemNombre  
    GROUP BY p.Id_Producto, p.EsPreparado  
)  
SELECT  
    dc.clientekey,  
    dm.mesakey,  
    de.empleadokey,  
    dp_dim.productokey,  
    dt.tiempokey,  
    p.Precio * dp.Cantidad AS ingresosVenta,  
    dp.Cantidad AS numeroVentas,
```



```

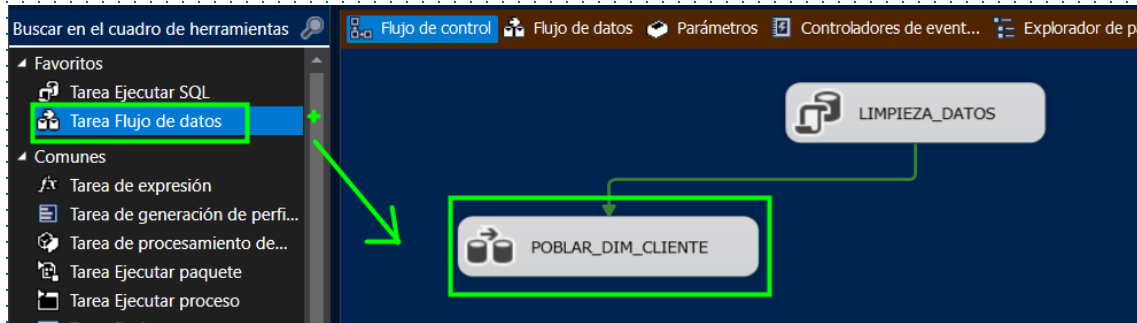
CASE WHEN pe.Estado = 'Completado' THEN 1 ELSE 0 END AS ventaCompletas,
dp.Cantidad * (
    CASE
        WHEN pc.EsPreparado = 1 THEN pc.CostoCalculadoIngredientes
        WHEN pc.EsPreparado = 0 THEN pc.CostoDirectoProducto
        ELSE 0
    END
) AS costo,
(p.Precio * dp.Cantidad) - (
    dp.Cantidad * (
        CASE
            WHEN pc.EsPreparado = 1 THEN pc.CostoCalculadoIngredientes
            WHEN pc.EsPreparado = 0 THEN pc.CostoDirectoProducto
            ELSE 0
        END
    )
) AS margen
FROM [RestauranteDBasePrueba5m].[TRANSACCION].[DetallePedido] dp
INNER JOIN [RestauranteDBasePrueba5m].[GENERAL].[Producto] p ON dp.Id_Producto = p.Id_Producto
INNER JOIN [RestauranteDBasePrueba5m].[GENERAL].[Categoria] cat ON p.Id_Categoria = cat.Id_Categoria
INNER JOIN [RestauranteDBasePrueba5m].[TRANSACCION].[Pedido] pe ON dp.Id_Pedido = pe.Id_Pedido
LEFT JOIN ProductoCosto pc ON dp.Id_Producto = pc.productoid
INNER JOIN [RestauranteDBasePrueba5m].[CLIENTE].[Cliente] c ON c.Id_Cliente = pe.Id_Cliente
INNER JOIN [RestauranteDBasePrueba5m].[PERSONAL].[Empleado] e ON e.Id_Empleado = pe.Id_Empleado
INNER JOIN [RestauranteDBasePrueba5m].[GENERAL].[Mesa] m ON m.Id_Mesa = pe.Id_Mesa
INNER JOIN RestauranteMart.DIM.Cliente dc ON (c.Nombres + ' ' + c.ApellidoPaterno + ' ' + c.ApellidoMaterno) = dc.nombres
INNER JOIN RestauranteMart.DIM.Mesa dm ON m.Ubicacion = dm.ubicacion AND m.Capacidad = dm.capacidad AND m.Estado = dm.estado
INNER JOIN RestauranteMart.DIM.Empleado de ON e.NombreCompleto = de.nombre
INNER JOIN RestauranteMart.DIM.Producto dp_dim ON p.Nombre = dp_dim.nombreProducto
                                AND cat.Nombre = dp_dim.categoria
                                AND p.EsPreparado = dp_dim.tipo
                                AND p.Precio = dp_dim.precio
INNER JOIN RestauranteMart.DIM.Tiempo dt ON CAST(pe.Fecha AS DATE) = CAST(dt.fecha AS DATE) -
ORDER BY dt.tiempokey;

```

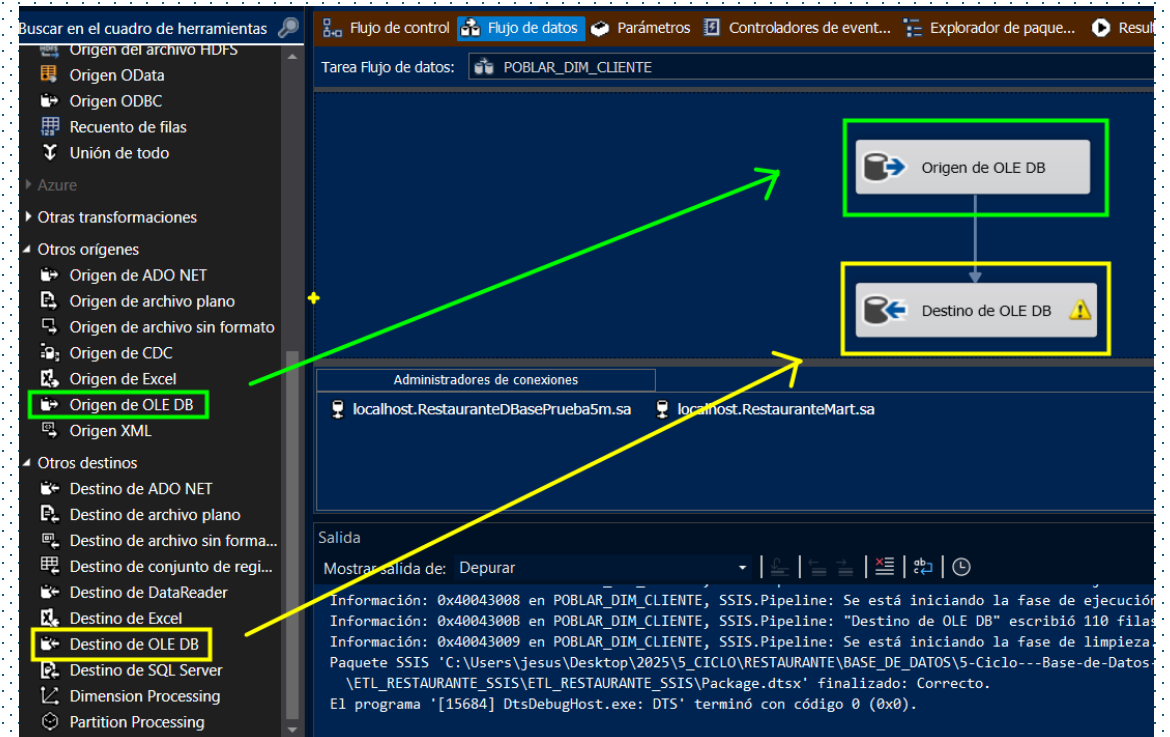
Carga de datos a dimensiones

Dimensión 1: Cliente

1. Creamos un flujo de datos para crear la dimensión Cliente:



2. Creamos el origen y el destino y configuramos la consulta y las filas:



Configure the properties used by a data flow to obtain data from any OLE DB provider.

Administrador de conexiones OLE DB:
localhost.RestauranteDBasePrueba5m.sa

Modo de acceso a datos:
Comando SQL

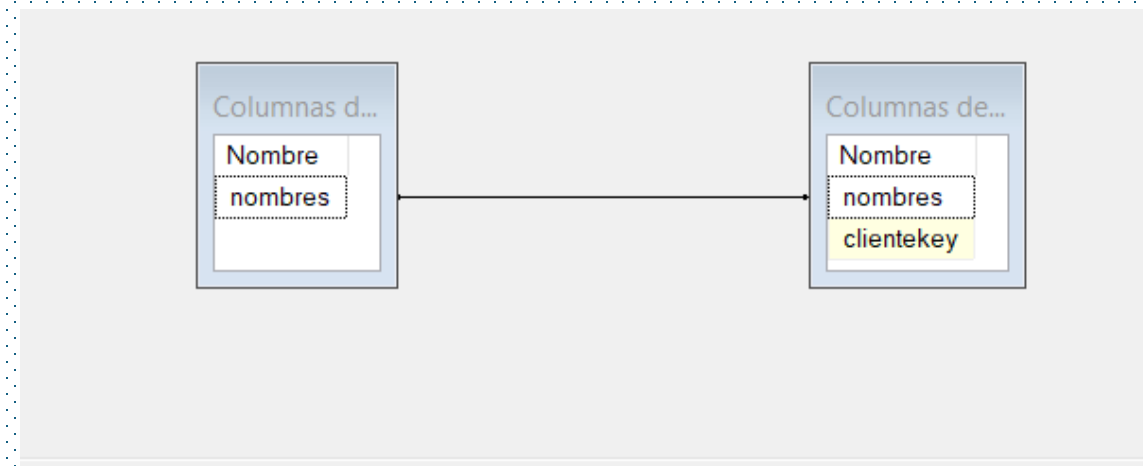
Texto de comando SQL:

```
SELECT
    (Nombres + ' ' + ApellidoPaterno + ' ' +
    ApellidoMaterno) AS nombres
FROM CLIENTE.Cliente
ORDER BY 1;
```

Vista previa de los resultados de la consulta (hasta las primeras 100 filas)

Resultados de la consulta (hasta las primeras 100 filas)

nombres
Adriana...
Alejand...
Alejand...
Alex Gó...
Ana Gar...
Ana Ma...
Ana Val...
Andrea ...
Andrea ...
Andrés ...
Andrés ...
Arturo ...
Arturo ...
Brenda ...
Bruno ...
Camila ...



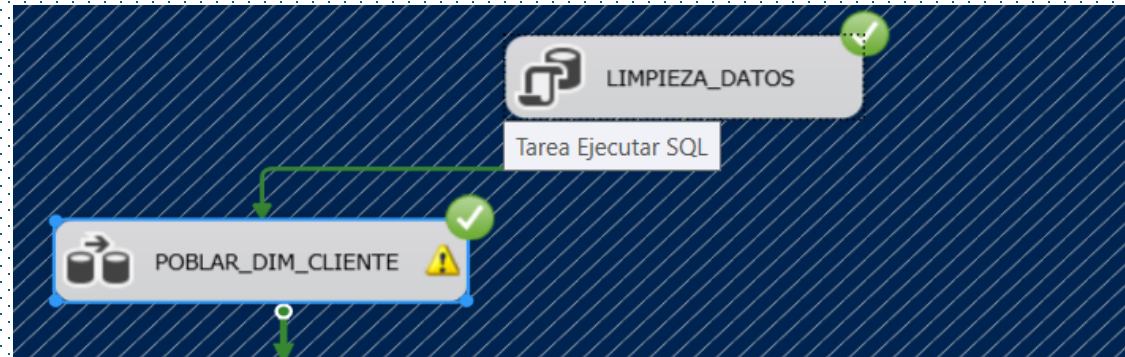
Columna de entrada	Columna de destino
nombres	nombres
omitir>	clientekey

Administrador de conexiones OLE DB:
localhost.RestauranteMart.sa

Modo de acceso a datos:
Tabla o vista

Nombre de la tabla o la vista:
[DIM].[Cliente]

3. Ejecutamos para poblar DIM.Cliente:



19
20

```
SELECT * FROM [RestauranteMart].DIM.CLIENTE
```

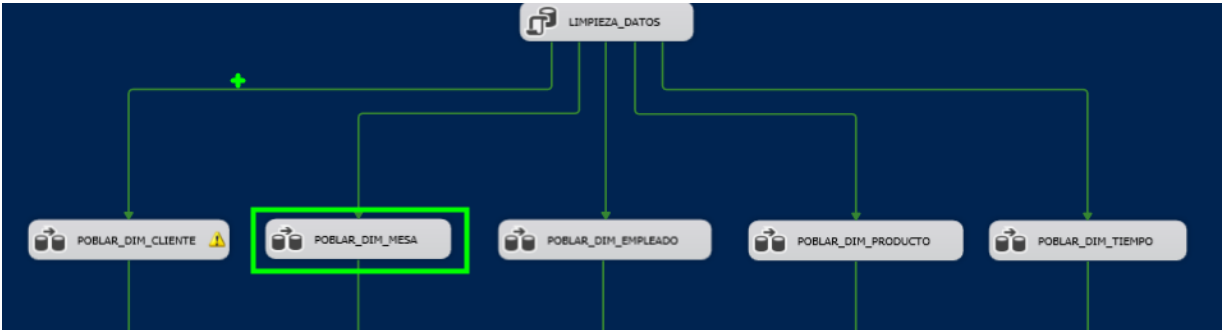
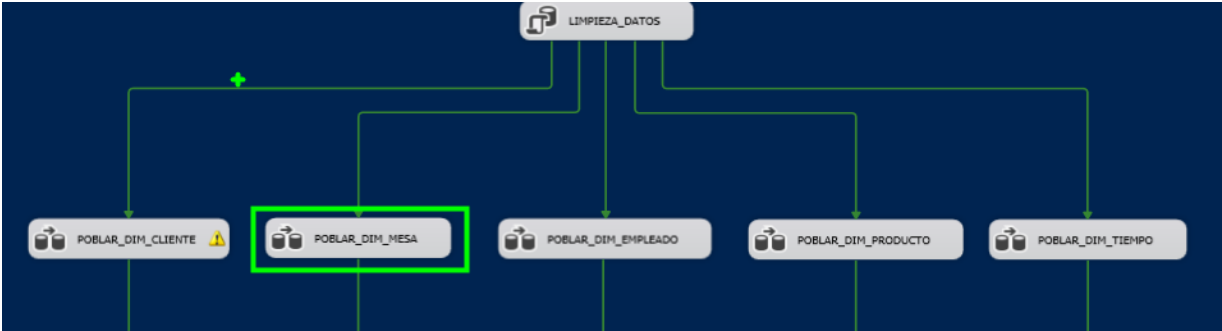
87 %

Results Messages

	clientekey	nombres
95	95	Roberto Vega Guerrero
96	96	Rosa Mayra Torres

Dimensión 2: Mesa

El procedimiento es el mismo para cada dimensión. Se crea el Flujo de datos, el origen y el destino, se prueba la consulta y se realiza el proceso de carga.



18
19
20

```
SELECT * FROM DIM.Mesa
```

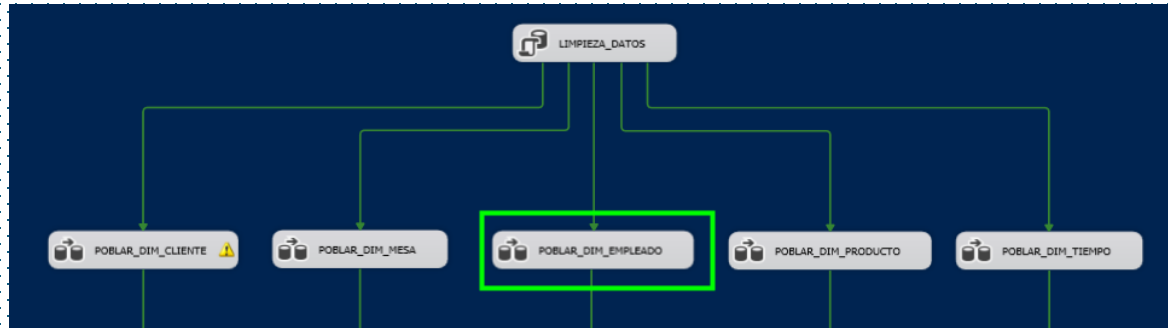
87 %

Results Messages

	mesakey	ubicacion	capacidad	estado
1	1	Primer Piso	4	Disponible
2	2	Primer Piso	4	Disponible
3	3	Primer Piso	4	Disponible
4	4	Primer Piso	4	Disponible
5	5	Primer Piso	4	Disponible

Dimensión 3: Empleado

El procedimiento es el mismo para cada dimensión. Se crea el Flujo de datos, el origen y el destino, se prueba la consulta y se realiza el proceso de carga.



19
20

```
SELECT * FROM DIM.Empleado
```

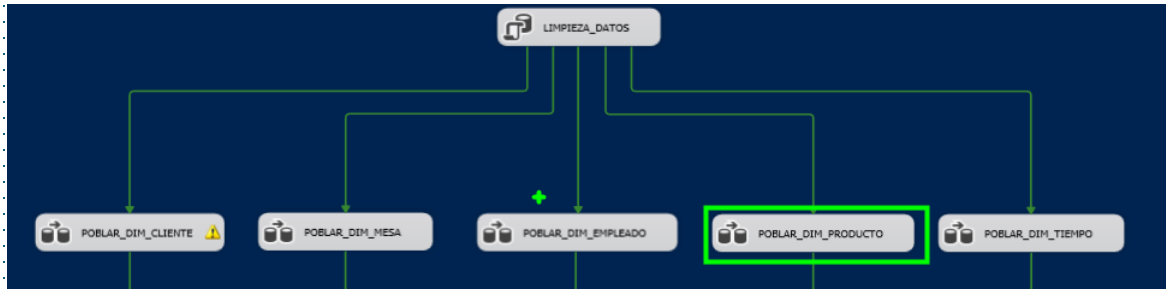
87 %

Results Messages

	empleadokey	nombre	fechaContratacion
1	1	Juan Perez Gonzalez	2022-01-10
2	2	Maria Lopez Torres	2022-03-01
3	3	Carlos Rodriguez Diaz	2022-06-15
4	4	Pedro Sanchez Flores	2021-07-01
5	5	Ana Gutierrez Vargas	2022-09-01
6	6	Sofia Mendoza Rojas	2022-02-20
7	7	José Carlos Álvarez	2021-01-01

Dimensión 4: Producto

El procedimiento es el mismo para cada dimensión. Se crea el Flujo de datos, el origen y el destino, se prueba la consulta y se realiza el proceso de carga.



19
20

```
SELECT * FROM DIM.Producto
```

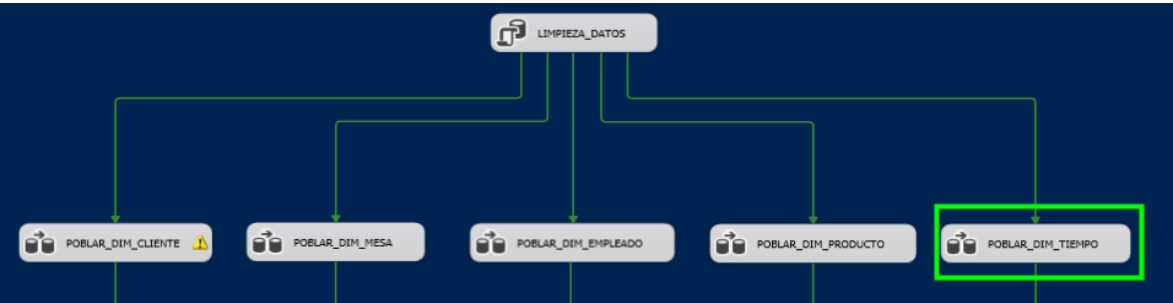
87 %

Results Messages

	productokey	categoria	tipo	precio	nombreProducto
1	1	Platos Típicos	1	28.00	1/4 Cuy Frito
2	2	Platos Típicos	1	28.00	1/4 Cuy Estofado
3	3	Platos Típicos	1	80.00	Cuy entero frito
4	4	Platos Típicos	1	80.00	Cuy entero estofado
5	5	Platos Típicos	1	25.00	Porción de hígados con mote
6	6	Platos Típicos	1	25.00	Chicharrones

Dimensión Tiempo

El procedimiento es el mismo para cada dimensión. Se crea el Flujo de datos, el origen y el destino, se prueba la consulta y se realiza el proceso de carga.



19
28
SELECT * FROM DIM.Tiempo

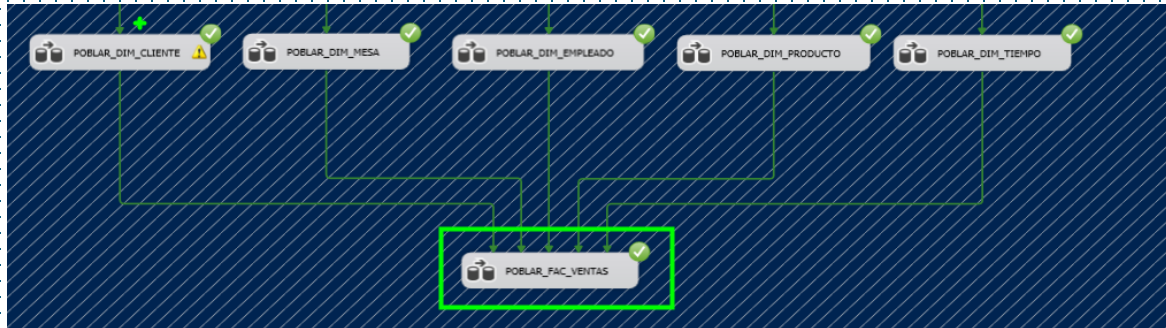
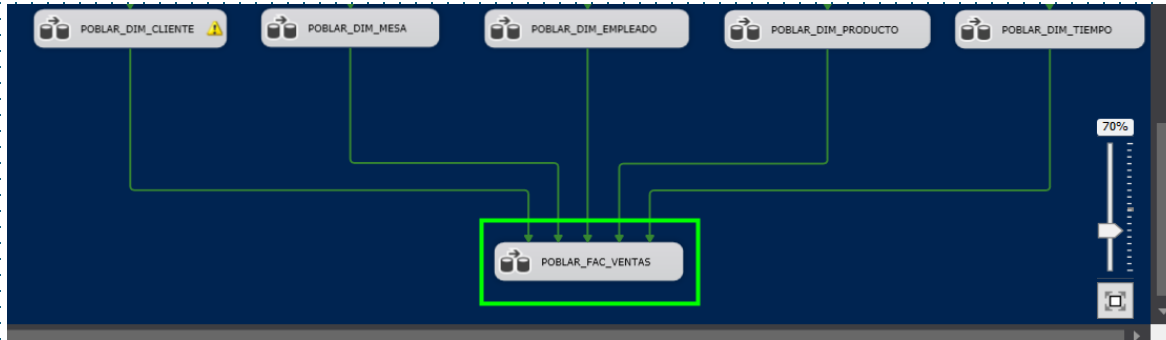
87 %

Results Messages

	tiempokey	fecha	anio	trimestre	nombreTrimestre	mes	nombreMes	semanaAnio	diaAnio	diaMes	diaSemana	nombreDia
1	1	2025-02-01 00:00:00.000	2025	1	Q1	2	Febrero	5	32	1	6	Sábado
2	2	2025-02-02 00:00:00.000	2025	1	Q1	2	Febrero	5	33	2	7	Domingo
3	3	2025-02-03 00:00:00.000	2025	1	Q1	2	Febrero	6	34	3	1	Lunes
4	4	2025-02-04 00:00:00.000	2025	1	Q1	2	Febrero	6	35	4	2	Martes
5	5	2025-02-05 00:00:00.000	2025	1	Q1	2	Febrero	6	36	5	3	Miércoles
6	6	2025-02-06 00:00:00.000	2025	1	Q1	2	Febrero	6	37	6	4	Jueves

Carga de datos a tabla hechos

El procedimiento es el mismo para la tabla de hechos. Se crea el Flujo de datos, el origen y el destino, se prueba la consulta y se realiza el proceso de carga.



20 | `SELECT * FROM FactVentas`

17 %

Results Messages

	ventakey	clientekey	mesakey	empleadokey	productokey	tiempokey	ingresosVenta	numeroVentas	ventaCompletas	costo	margen
1	1	57	1	1	1	1	28,00	1	1	5,84	22,16
2	2	57	2	1	1	1	28,00	1	1	5,84	22,16
3	3	57	3	1	1	1	28,00	1	1	5,84	22,16
4	4	57	4	1	1	1	28,00	1	1	5,84	22,16
5	5	57	5	1	1	1	28,00	1	1	5,84	22,16
6	6	57	6	1	1	1	28,00	1	1	5,84	22,16

ETL con Python

1. Importación de librerías

```
import pyodbc # Librería para conectar Python con bases de datos SQL Server
import pandas as pd # Librería para manipular datos en forma de tablas
(DataFrames)
```

2. Conexión a base de datos transaccional (RestauranteDBasePrueba5m)

Se conecta a la base de datos original (transaccional), donde están los datos en bruto como clientes, empleados, productos, etc.

```
# Conexión a la base de datos transaccional donde están los datos operativos
try:
    conn_nw = pyodbc.connect('DRIVER={SQL
Server};SERVER=IDEAPAD\SQL;DATABASE=RestauranteDBasePrueba5m;UID=sa;PWD=gnab')
    cursor_nw = conn_nw.cursor()
    print('Conexión a RestauranteDBasePrueba5m correcta.')
except Exception as e:
    print('Conexión a RestauranteDBasePrueba5m fallida:', e)
    exit()
```

ETL con Python

3. Consultas SQL para extraer datos

query1 hasta query5: consultas para crear las dimensiones Cliente, Mesa, Empleado, Producto y Tiempo.

Consulta para obtener nombres completos de los clientes

```
query1 = """
SELECT
    (Nombres + ' ' + ApellidoPaterno + ' ' + ApellidoMaterno) AS nombres
FROM CLIENTE.Cliente
ORDER BY 1;
"""
```

4. Ejecucion de consultas y carga a DataFrames

Cada resultado de consulta se carga en una tabla en memoria usando pandas. Para transformarlos fácilmente antes de enviarlos al Data Warehouse.

```
df1 = pd.read_sql(query1, conn_nw)
df2 = pd.read_sql(query2, conn_nw)
df3 = pd.read_sql(query3, conn_nw)
df4 = pd.read_sql(query4, conn_nw)
df5 = pd.read_sql(query5, conn_nw)
df = pd.read_sql(query, conn_nw)
```

ETL con Python

5. Transformación: convertir fechas

Convierte las fechas a un tipo reconocido por Python para trabajar con ellas más fácilmente

```
print("----- TRANSFORMACIÓN DE DATOS -----")
df5['fecha'] = pd.to_datetime(df5['fecha']) # Convierte texto a fecha real
```

6. Limpieza: eliminar duplicados

Se limpian las tablas para que no se inserten datos duplicados en el Data Warehouse.

```
dcliente = df1[['nombres']].drop_duplicates()
dmesa = df2[['ubicacion', 'capacidad', 'estado']]
dempleado = df3[['nombre', 'fechaContratacion']].drop_duplicates()
dproducto = df4[['categoria', 'tipo', 'precio', 'nombreProducto']].drop_duplicates()
dtiempo = df5[['fecha', 'anio', 'trimestre', 'nombreTrimestre', 'mes', 'nombreMes',
'semanaAnio', 'diaAnio', 'diaMes', 'diaSemana', 'nombreDia']].drop_duplicates()
```


ETL con Python

7. Conexión al Data Warehouse (RestauranteMart)

Se conecta al Data Warehouse, que es el destino donde se cargarán los datos ya transformados.

```
print("----- CONEXIÓN A BASE DE DATOS DIMENSIONAL -----")
try:
    conn_dw = pyodbc.connect('DRIVER={SQL
Server};SERVER=IDEAPAD\SQL;DATABASE=RestauranteMart;UID=sa;PWD=gnab')
    cursor_dw = conn_dw.cursor()
    print('Conexión a RestauranteMart correcta.')
except Exception as e:
    print('Conexión a RestauranteMart fallida:', e)
    exit()
```

8. Limpieza previa de tablas en el DW

Elimina todos los datos actuales del DW y reinicia los contadores de claves primarias (IDENTITY) para empezar desde cero.

```
cursor_dw.execute("DELETE FROM dbo.FactVentas;")
tablas = [ 'Cliente', 'Mesa', 'Empleado', 'Producto', 'Tiempo' ]
for tabla in tablas:
    cursor_dw.execute(f"DELETE FROM DIM.{tabla};")
    cursor_dw.execute(f"DBCC CHECKIDENT ('DIM.{tabla}', RESEED, 0);")
    conn_dw.commit()
cursor_dw.execute("DBCC CHECKIDENT ('dbo.FactVentas', RESEED, 0);")
```

ETL con Python

9. Carga de datos en tablas de dimensiones

Inserta fila por fila en las tablas de dimensiones: Cliente, Mesa, Empleado, Producto y Tiempo.

```
# Cargar datos a tabla DIM.Cliente
for _, row in dcliente.iterrows():
    cursor_dw.execute("INSERT INTO DIM.Cliente (nombres) VALUES (?)", row.nombres)
conn_dw.commit()
print("Carga de dimensión Cliente completada")
```

10. Carga de tabla de hechos (FactVentas)

Inserta los registros de ventas, que enlazan claves de dimensión y métricas como ingresos, costos, número de ventas, etc.

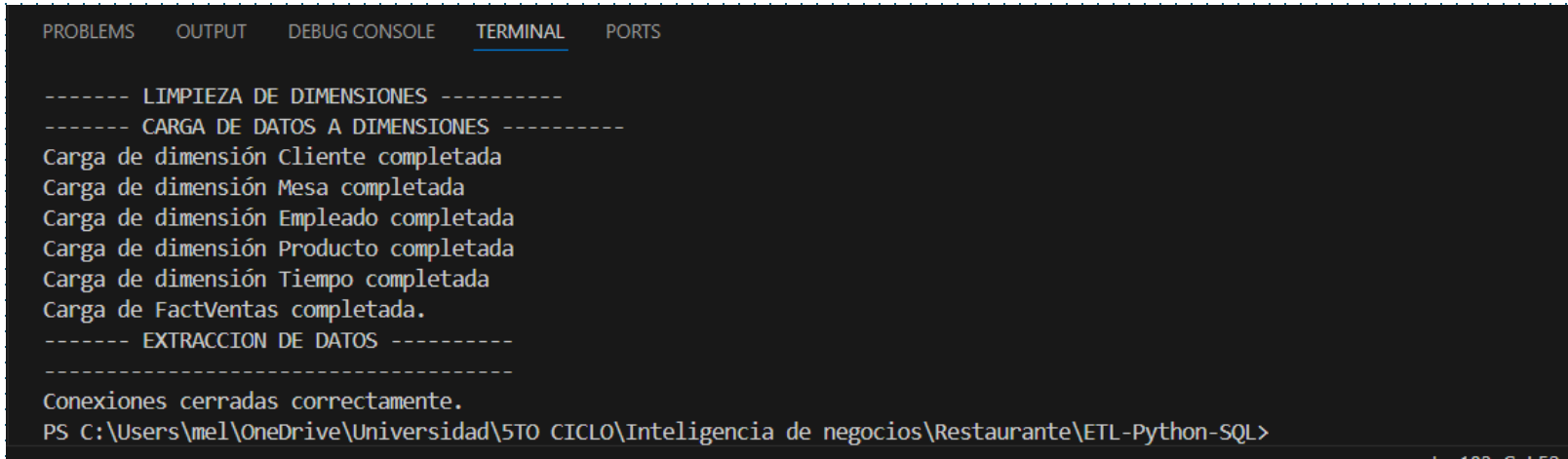
```
for _, row in df.iterrows():
    cursor_dw.execute("""
        INSERT INTO dbo.FactVentas (
            clientekey, mesakey, empleadokey, productokey,
            tiempokey, ingresosVenta, numeroVentas, ventaCompletas, costo, margen
        ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
        row.clientekey, row.mesakey, row.empleadokey, row.productokey,
        row.tiempokey, row.ingresosVenta, row.numeroVentas, row.ventaCompletas,
        row.costo, row.margen)
conn_dw.commit()
print('Carga de FactVentas completada.')
```

ETL con Python

11. Cierre de conexiones

```
cursor_dw.close()  
conn_dw.close()  
conn_nw.close()  
print('Conexiones cerradas correctamente.')
```

12. Resultado en Terminal



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
  
----- LIMPIEZA DE DIMENSIONES -----  
----- CARGA DE DATOS A DIMENSIONES -----  
Carga de dimensión Cliente completada  
Carga de dimensión Mesa completada  
Carga de dimensión Empleado completada  
Carga de dimensión Producto completada  
Carga de dimensión Tiempo completada  
Carga de FactVentas completada.  
----- EXTRACCION DE DATOS -----  
-----  
Conexiones cerradas correctamente.  
PS C:\Users\mel\OneDrive\Universidad\5TO CICLO\Inteligencia de negocios\Restaurante\ETL-Python-SQL>
```

Scrip_DB_Dimensional.sql - IDEAPAD\SQL.RestauranteMart (IDEAPAD\mel (54))* - Microsoft SQL Server Management Studio

Inicio rápido (Ctrl+Q)

Archivo Editar Ver Consulta Proyecto Herramientas Ventana Ayuda

RestauranteMart

Ejecutar

Explorador de objetos

Conectar

IDEAPAD\SQL (Microsoft Analysis)

- Bases de datos
- Ensamblados
- Administración

IDEAPAD\SQL (16.0.1135.2 de SQL)

Scrip_DB_Dimensio...IDEAPAD\mel (54))*CONSULTAS_MULTI...DEAPAD\mel (63))IdeaPad\SQLResta...teMart - Diagram_0*SQLQuery2.sql - ID...(IDEAPAD\mel (87))

```
USE RestauranteMart;  
SELECT * FROM DIM.Cliente  
SELECT * FROM DIM.Empleado  
SELECT * FROM DIM.Mesa  
SELECT * FROM DIM.Producto  
SELECT * FROM DIM.Tiempo  
SELECT * FROM dbo.FactVentas
```

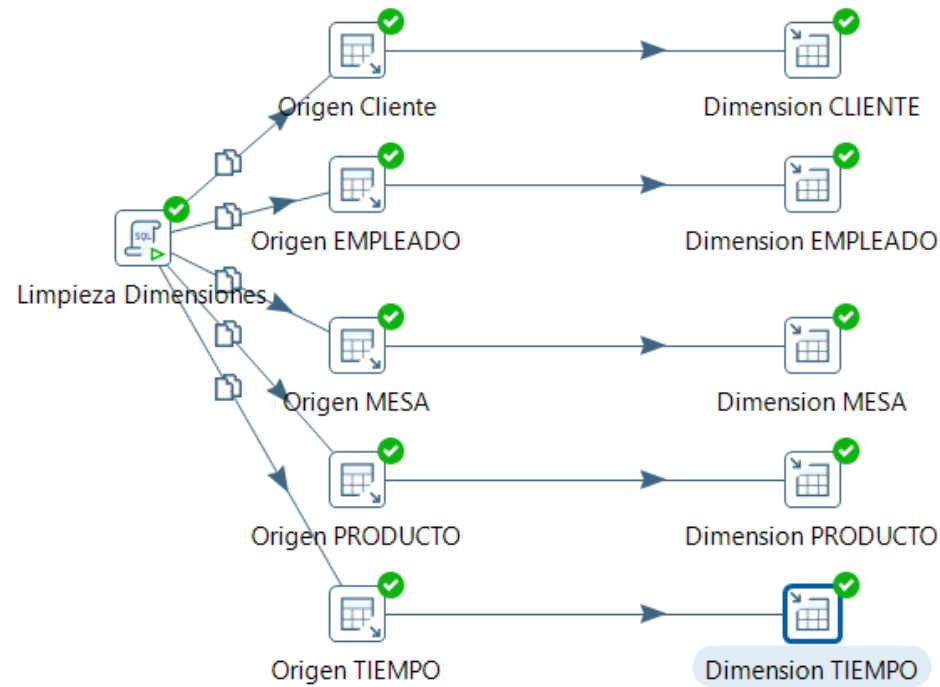
121 %

Resultados Mensajes

	venta	cliente	mesa	empleado	producto	tiempo	ingreso	numero	venta	costo	margen
	key	key	key	key	key	key	Venta	Ventas	Completas		
92...	9206	79	8	2	5	143	50,00	2	1	6,50	43,50
92...	9207	79	9	2	5	143	50,00	2	1	6,50	43,50
92...	9208	79	10	2	5	143	50,00	2	1	6,50	43,50
92...	9209	79	11	2	5	143	50,00	2	1	6,50	43,50
92...	9210	79	12	2	5	143	50,00	2	1	6,50	43,50
92...	9211	79	1	2	35	143	20,00	1	1	15,...	5,00
92...	9212	79	2	2	35	143	20,00	1	1	15,...	5,00
92...	9213	79	3	2	35	143	20,00	1	1	15,...	5,00
92...	9214	79	4	2	35	143	20,00	1	1	15,...	5,00
92...	9215	79	5	2	35	143	20,00	1	1	15,...	5,00
92...	9216	79	6	2	35	143	20,00	1	1	15,...	5,00
92...	9217	79	13	2	35	143	20,00	1	1	15,...	5,00
92...	9218	79	14	2	35	143	20,00	1	1	15,...	5,00
92...	9219	79	15	2	35	143	20,00	1	1	15,...	5,00
92...	9220	79	7	2	35	143	20,00	1	1	15,...	5,00
92...	9221	79	8	2	35	143	20,00	1	1	15,...	5,00
92...	9222	79	9	2	35	143	20,00	1	1	15,...	5,00
92...	9223	79	10	2	35	143	20,00	1	1	15,...	5,00

ETL con Python

ETL con Pentaho (Opcional)



Execution Results

Logging Execution History Step Metrics Performance Graph Metrics Preview data



2025/07/05 10:15:01 - Dimension MESA.0 - Procesamiento finalizado (I=0, O=37, R=37, W=37, U=0, E=0)

2025/07/05 10:15:01 - Dimension CLIENTE.0 - Procesamiento finalizado (I=0, O=110, R=110, W=110, U=0, E=0)

2025/07/05 10:15:01 - Dimension PRODUCTO.0 - Procesamiento finalizado (I=0, O=48, R=48, W=48, U=0, E=0)

2025/07/05 10:15:01 - Dimension TIEMPO.0 - Procesamiento finalizado (I=0, O=150, R=150, W=150, U=0, E=0)

2025/07/05 10:15:01 - Spoon - La transformación ha finalizado!!

ETL con Pentaho (Opcional)

Poblar Dimensión TABLA FactVentas

Entrada Tabla

Nombre paso Consulta RestauranteMart

Conexión Origen_RestauranteDBase5mese

SQL

```
dp.Cantidad AS numeroVentas,
CASE WHEN pe.Estado = 'Completado' THEN 1 ELSE 0 END AS ventaCompletas,
dp.Cantidad * (
CASE
WHEN pc.EsPreparado = 1 THEN pc.CostoCalculadoIngredientes
WHEN pc.EsPreparado = 0 THEN pc.CostoDirectoProducto
ELSE 0 -- En caso de que EsPreparado tenga otro valor o sea NULL
END
) AS costo,
(p.Precio * dp.Cantidad) - (
dp.Cantidad * (
CASE
WHEN pc.EsPreparado = 1 THEN pc.CostoCalculadoIngredientes
WHEN pc.EsPreparado = 0 THEN pc.CostoDirectoProducto
ELSE 0
END
)
) AS aargen
FROM [RestauranteDBasePrueba5m].[TRANSACCION].[DetallePedido] dp
INNER JOIN [RestauranteDBasePrueba5m].[GENERAL].[Producto] p ON dp.Id_Producto = p.Id_Producto
--CORRECCION ** Añadir JOIN a la tabla Categoria para obtener el nombre de la categoria
INNER JOIN [RestauranteDBasePrueba5m].[GENERAL].[Categoria] cat ON p.Id_Categoria = cat.Id_Cate
INNER JOIN [RestauranteDBasePrueba5m].[TRANSACCION].[Pedido] pe ON dp.Id_Pedido = pe.Id_Pedido
LEFT JOIN ProductoCosto pc ON dp.Id_Producto = pc.productoid
INNER JOIN [RestauranteDBasePrueba5m].[CLIENTE].[Cliente] c ON c.Id_Cliente = pe.Id_Cliente
INNER JOIN [RestauranteDBasePrueba5m].[PERSONAL].[Empleado] e ON e.Id_Empleado = pe.Id_Empleado
INNER JOIN [RestauranteDBasePrueba5m].[GENERAL].[Mesa] m ON m.Id_Mesa = pe.Id_Mesa
INNER JOIN RestauranteMart.DIM.Cliente dc ON (c.Nombres + ' ' + c.ApellidoPaterno + ' ' + c.Ape
INNER JOIN RestauranteMart.DIM.Mesa da ON m.Ubicacion = da.ubicacion AND m.Capacidad = da.capac
INNER JOIN RestauranteMart.DIM.Empleado de ON e.NombreCompleto = de.nombre
INNER JOIN RestauranteMart.DIM.Producto dp_dia ON p.Nombre = dp_dia.nombreProducto
AND cat.Nombre = dp_dia.categoria
AND p.EsPreparado = dp_dia.tipo
AND p.Precio = dp_dia.precio
INNER JOIN RestauranteMart.DIM.Tiempo dt ON CAST(pe.Fecha AS DATE) = CAST(dt.fecha AS DATE) --
ORDER BY dt.tiempokey;
```

Line 74 Column 22

Store column info in step meta data ☐

Enable lazy conversion ☐

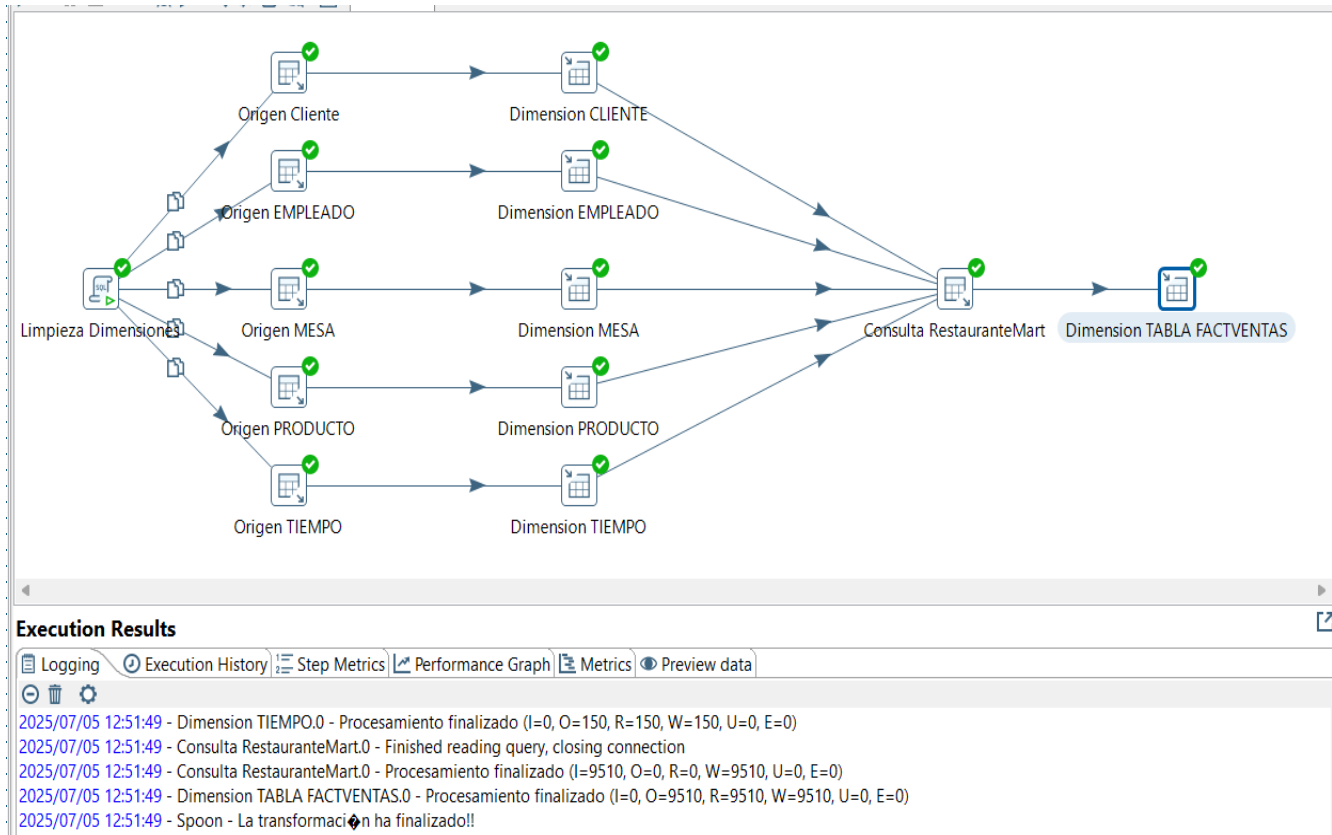
Reemplazar variables en script? ☐

Insertar datos del paso

Ejecutar para cada fila? ☐

Limitar tamaño 0

Help Vale Previsualizar Cancelar



ETL con Pentaho (Opcional)

Revisar los datos cargados a la DIMENSION TABLA FACTVENTAS

12
13
14

```
SELECT * FROM [dbo].[FactVentas]
```

107 %

Resultados Mensajes

	ventaKey	clienteKey	mesaKey	empleadoKey	productoKey	tiempoKey	ingresosVenta	numeroVentas	ventaCompletas	costo	margen
94...	9491	92	11	1	22	150	15,00	1	1	1,1...	13,855
94...	9492	92	12	1	22	150	15,00	1	1	1,1...	13,855
94...	9493	92	13	1	22	150	15,00	1	1	1,1...	13,855
94...	9494	92	14	1	22	150	15,00	1	1	1,1...	13,855
94...	9495	92	15	1	22	150	15,00	1	1	1,1...	13,855
94...	9496	92	1	1	34	150	12,00	1	1	5,00	7,00
94...	9497	92	2	1	34	150	12,00	1	1	5,00	7,00
94...	9498	92	3	1	34	150	12,00	1	1	5,00	7,00
94...	9499	92	4	1	34	150	12,00	1	1	5,00	7,00
95...	9500	92	5	1	34	150	12,00	1	1	5,00	7,00
95...	9501	92	6	1	34	150	12,00	1	1	5,00	7,00
95...	9502	92	7	1	34	150	12,00	1	1	5,00	7,00
95...	9503	92	8	1	34	150	12,00	1	1	5,00	7,00
95...	9504	92	9	1	34	150	12,00	1	1	5,00	7,00
95...	9505	92	10	1	34	150	12,00	1	1	5,00	7,00
95...	9506	92	11	1	34	150	12,00	1	1	5,00	7,00
95...	9507	92	12	1	34	150	12,00	1	1	5,00	7,00
95...	9508	92	13	1	34	150	12,00	1	1	5,00	7,00
95...	9509	92	14	1	34	150	12,00	1	1	5,00	7,00
95...	9510	92	15	1	34	150	12,00	1	1	5,00	7,00

Consulta ejecutada correctamente.

GRACIAS

